

What is SQL?

SQL (Structured Query Language) is a standard programming language for interacting with relational database systems. It allows users to perform a wide range of operations, such as creating and managing database objects, manipulating data, and controlling access.

Types of SQL Statements

SQL statements are categorized based on their functionalities. Each category handles specific aspects of database operations.

1. Data Definition Language (DDL)

DDL statements are used to define, alter, or delete the structure of database objects such as tables, views, and indexes.

Key DDL Operations

1. **CREATE:** Create database objects like tables or indexes.
2. **ALTER:** Modify existing database structures.
3. **DROP:** Remove database objects.
4. **TRUNCATE:** Remove all rows from a table without logging individual row deletions.

Examples

1. **Create a database:**
CREATE DATABASE Practice;
2. **Create a Table:**
CREATE TABLE Employees (
 EmployeeID INT PRIMARY KEY,
 Name VARCHAR(50),
 DOB DATE,
 Salary DECIMAL(10, 2)
);
3. **Alter a Table:**

- **Add a Column:**
ALTER TABLE Employees ADD Department VARCHAR(50);
- **Remove a Column:**
ALTER TABLE Employees DROP COLUMN Department;
- **Rename a Column:**
ALTER TABLE Employees RENAME COLUMN Salary TO MonthlySalary;
- **Change Data Type:**

ALTER TABLE Employees MODIFY COLUMN MonthlySalary
FLOAT;

4. Truncate a Table:

TRUNCATE TABLE Employees;

- Removes all rows in the table without logging individual deletions. It resets the auto-increment counter, making it faster than DELETE.

5. Drop a Table:

DROP TABLE Employees;

2. Data Manipulation Language (DML)

DML statements are used to manipulate data within tables. These include inserting, updating, deleting, and querying data.

Key DML Operations

1. **INSERT:** Add new records.
2. **UPDATE:** Modify existing records.
3. **DELETE:** Remove specific records.
4. **REPLACE:** Replace a record (delete old and insert new if a primary key conflict occurs).

Examples

1. **Insert Data:**

INSERT INTO Employees (EmployeeID, Name, DOB, Salary)

VALUES (1, 'John Doe', '1990-01-15', 50000);

2. Update Data:

UPDATE Employees SET Salary = (Salary*1.1) WHERE EmployeeID = 1;

3. Delete Data:

DELETE FROM Employees WHERE EmployeeID = 1;

4. Replace Data:

**REPLACE INTO Employees (EmployeeID, Name, DOB, Salary)
VALUES (1, 'Jane Doe', '1992-05-20', 60000);**

5. Select Data:

SELECT Name, Salary FROM Employees WHERE DOB > '1990-01-01';

3. Data Query Language (DQL)

DQL consists primarily of the SELECT statement, which is used to query data.

Examples

1. Basic Select:

SELECT * FROM Employees;

2. Filtered Select:

SELECT Name, DOB FROM Employees WHERE Salary > 50000;

3. Aggregated Query:

SELECT Department, AVG(Salary) AS "Average Salary of Each Department" FROM Employees GROUP BY Department;

4. Join Query:

**SELECT E.Name, D.DepartmentName
FROM Employees E INNER JOIN Departments D**

ON E.DepartmentID = D.DepartmentID;

4. Data Control Language (DCL)

DCL manages access to the database and its objects.

Key DCL Operations

1. **GRANT:** Assign privileges to users.
2. **REVOKE:** Remove privileges from users.

Examples

CREATE USER 'user1'@'localhost' IDENTIFIED BY 'password';

Check users

SELECT User, Host FROM mysql.user;

SHOW GRANTS FOR 'user1'@'localhost'; -- to show permissions

1. Grant Privileges:

GRANT SELECT, INSERT ON Practice.Employees TO 'user1'@'localhost';

FLUSH PRIVILEGES -- For reloading privilege table to apply changes

SHOW GRANTS FOR 'user1'@'localhost'; -- to show permissions

Note : Now you can logout from root user and login user1 and can use Practice Database Employee table to insert and select record.

2. Revoke Privileges:

REVOKE SELECT ON Employees FROM 'user1'@'localhost';

SHOW GRANTS FOR 'user1'@'localhost'; -- to show permissions

To Delete created users

DROP USER 'user1'@'localhost';

5. Transaction Control Language (TCL)

TCL manages database transactions, ensuring data consistency and integrity.

Key TCL Operations

1. **COMMIT**: Save changes permanently.
2. **ROLLBACK**: Undo changes.
3. **SAVEPOINT**: Set a point in a transaction to which you can later roll back.

Examples

1. **Transaction Management:**
START TRANSACTION;

UPDATE Employees SET Salary = 60000 WHERE EmployeeID = 2;

ROLLBACK; -- Undo the update
2. **Savepoint Example:**
SAVEPOINT sp1;

UPDATE Employees SET Salary = 70000 WHERE EmployeeID = 3;

ROLLBACK TO sp1; -- Undo changes since the savepoint

COMMIT; -- Save changes permanently

Summary of SQL Types

Type	Purpose	Examples
DDL	Define or modify database structures	CREATE, ALTER, DROP, TRUNCATE
DML	Manipulate data	INSERT, UPDATE, DELETE, REPLACE
DQL	Query data	SELECT, JOIN, GROUP BY
DCL	Control access	GRANT, REVOKE
TCL	Manage transactions	COMMIT, ROLLBACK,

		SAVEPOINT
--	--	-----------

Calculating Age (Derived Attribute) not included while creating tables.

```
SELECT Name, TIMESTAMPDIFF(YEAR, DOB, CURDATE()) AS Age
FROM Employees;
```

More Accurate :

```
SELECT Name, TIMESTAMPDIFF(YEAR, DOB, CURDATE()) -
  (DATE_FORMAT(CURDATE(), '%m%d') < DATE_FORMAT(DOB, '%m%d')) AS Age
FROM Employees;
```

Note : This query subtracts 1 if the current date (%m%d) is earlier than the birth date (%m%d) in the current year.

Suppose we have a table Employees:

```
CREATE TABLE Employees (
  EmployeeID INT PRIMARY KEY,
  Name VARCHAR(50),
  DOB DATE
);
```

Insert Sample Data

```
INSERT INTO Employees (EmployeeID, Name, DOB) VALUES
(1, 'Alice', '1990-05-15'),
(2, 'Bob', '1985-12-10'),
(3, 'Charlie', '2000-08-25');
```

Output

Name	Age
Alice	34
Bob	38
Charlie	24

Run the Query

```
SELECT Name, TIMESTAMPDIFF(YEAR, DOB, CURDATE()) AS Age FROM
Employees;
```

Open shell in xampp
login to mysql using command `mysql -u root -p` and press enter and when password is prompted then again press enter since there is no password by default

Create a database CollegeDB

Create 7 tables :

1) StuInfo

```
CREATE TABLE StuInfo (  
Sid INT PRIMARY KEY AUTO_INCREMENT,  
SName VARCHAR(50) NOT NULL,  
Stream VARCHAR(4) DEFAULT 'BCA' CHECK (Stream IN ('BCA', 'BBA',  
'MCA', 'MBA', 'BLLB')),  
SAddrS VARCHAR(100) NOT NULL,  
Semester VARCHAR(4) CHECK (Semester IN ('I', 'II', 'III', 'IV', 'V', 'VI', 'VII',  
'VIII')),  
Gender VARCHAR(1) DEFAULT 'M' CHECK (Gender IN ('M', 'F', 'O')),  
DOB DATE NOT NULL,  
MobNo CHAR(10) UNIQUE CHECK (MobNo REGEXP '^[0-9]{10}$'),  
Email VARCHAR(255) UNIQUE  
);
```

2) FeeInfo

```
CREATE TABLE FeeInfo (  
Sid INT,  
Semester VARCHAR(2),  
Amount DECIMAL(10, 2),  
Paid CHAR(1) DEFAULT 'N' CHECK (Paid IN ('Y', 'N')),  
PRIMARY KEY (Sid, Semester),  
FOREIGN KEY (Sid) REFERENCES StuInfo(Sid),  
FOREIGN KEY (Semester) REFERENCES StuInfo(Semester)  
ON UPDATE CASCADE  
ON DELETE CASCADE  
);
```

3) BookInfo

```
CREATE TABLE BookInfo (  
BookId INT AUTO_INCREMENT PRIMARY KEY,  
Title VARCHAR(255) NOT NULL,
```

```
Genre VARCHAR(100),  
Publisher VARCHAR(255),  
Author VARCHAR(255) NOT NULL,  
PublicationDate DATE,  
ISBN VARCHAR(13) UNIQUE,  
Pages INT CHECK (Pages > 0),  
Language VARCHAR(25) DEFAULT 'English',  
Price DECIMAL(8, 2) CHECK (Price >= 0)  
);
```

4) BookBorrowInfo

```
CREATE BookBorrowInfo(  
BorrowId INT AUTO_INCREMENT PRIMARY KEY,  
Sid INT,  
BookId INT,  
BorrowDate DATE DEFAULT CURRENT_DATE,  
ToReturnDate DATE,  
FOREIGN KEY (Sid) REFERENCES StuInfo(Sid),  
FOREIGN KEY (BookId) REFERENCES BookInfo(BookId)  
);
```

5) BookReturnInfo

```
CREATE BookReturnInfo(  
ReturnId INT AUTO_INCREMENT PRIMARY KEY,  
BorrowId INT,  
ReturnDate DATE DEFAULT CURRENT_DATE,  
FOREIGN KEY (BorrowId) REFERENCES BookBorrowInfo(BorrowId)  
);
```

6) EmplInfo

```
CREATE TABLE EmplInfo (  
EId INT AUTO_INCREMENT PRIMARY KEY,  
ENAME VARCHAR(100),  
Address VARCHAR(255),  
DOB DATE,  
Gender CHAR(1) DEFAULT 'M' CHECK (Gender IN ('M', 'F', 'O')),
```

JoinDate DATE NOT NULL,
 Qualification VARCHAR(25),
 Post VARCHAR(25),
 DeptId INT,
 MgrId INT,
 Salary DECIMAL(8, 2),
 MobNo CHAR(10) UNIQUE CHECK (MobNo REGEXP '^[0-9]{10}\$'),
 Email VARCHAR(255) NOT NULL UNIQUE,
 FOREIGN KEY (DeptId) REFERENCES DeptInfo(DeptId),
 FOREIGN KEY (MgrId) REFERENCES EmpInfo(EId)
);

7) DeptInfo

CREATE TABLE DeptInfo(
 DeptId INT PRIMARY KEY AUTO_INCREMENT,
 DeptName VARCHAR(25) NOT NULL,
 DeptHead VARCHAR(100) NOT NULL,
 DeptLocation VARCHAR(100)
);

Creating users

Syntax : CREATE USER 'username'@'host' IDENTIFIED BY 'password';

example 1 : CREATE USER 'jrc'@'localhost' IDENTIFIED BY 'jrc';

example 2 : CREATE USER 'drg'@'localhost' IDENTIFIED BY 'drg';

example 3 : CREATE USER 'gm'@'localhost' IDENTIFIED BY 'gm';

example 4 : CREATE USER 'ram'@'localhost' IDENTIFIED BY 'ram';

example 5 : CREATE USER 'sita'@'localhost' IDENTIFIED BY 'sita';

example 6 : CREATE USER 'laxman'@'localhost' IDENTIFIED BY 'laxman';

example 7 : CREATE USER 'bharat'@'localhost' IDENTIFIED BY 'bharat';

example 8 : CREATE USER 'satrudhan'@'localhost' IDENTIFIED BY 'satrudhan';

example 9 : CREATE USER 'urmila'@'localhost' IDENTIFIED BY 'urmila';

example 10 : CREATE USER 'libuser1'@'localhost' IDENTIFIED BY 'libuser1';

To see all the users

SELECT User, Host FROM mysql.user;

To See currently logged in user

SELECT USER();
 SELECT CURRENT_USER();

To see all the created databases and tables

SHOW DATABASES;
 SHOW TABLES FROM database_name;

If you want to work inside a database

USE database_name;

To see a particular user with grants given to him

SHOW GRANTS FOR 'jrc'@'localhost';

Granting privileges to the created user

Granting All privileges Syntax :

GRANT ALL PRIVILEGES ON database_name.* TO 'username'@'host';

Granting all privileges including grant option :

GRANT ALL PRIVILEGES ON *.* TO 'username'@'host' WITH GRANT OPTION;

Granting all privileges on a specific database

GRANT ALL PRIVILEGES ON CollegeDB.* TO 'gm'@'localhost';

Granting all privileges on all database and all object with grant options

GRANT ALL PRIVILEGES ON *.* TO 'jrc'@'localhost' WITH GRANT OPTION;

Granting all privileges to specific table in a database

GRANT ALL PRIVILEGES ON CollegeDB.StuInfo TO 'ram'@'localhost';
 GRANT ALL PRIVILEGES ON CollegeDB.EmpInfo TO 'ram'@'localhost';

Granting Privilege to create database and objects like table, view, routine, trigger etc

GRANT CREATE ON *.* TO 'laxman'@'localhost';

Granting Privilege to create objects like table, view, routine, trigger etc inside CollegeDB

GRANT CREATE ON CollegeDB.* TO 'laxman'@'localhost';

Granting various Operation on StuInfo, EmpInfo and BookInfo table to Bharat

```
GRANT SELECT, INSERT, UPDATE, DELETE ON CollegeDB.StuInfo TO 'bharat'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON CollegeDB.EmplInfo TO 'bharat'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON CollegeDB.BookInfo TO 'bharat'@'localhost';
```

```
Granting various operation on tables to Urmila
GRANT SELECT ON CollegeDB.* TO 'urmila'@'localhost'
GRANT INSERT ON CollegeDB.BookInfo TO 'urmila'@'localhost'
GRANT INSERT, UPDATE, DELETE ON CollegeDB.BookBorrowInfo TO 'urmila'@'localhost'
```

After granting privileges we need to reload privilege table to apply changes:

```
FLUSH PRIVILEGES;
```

To see a particular user with grants given to him(Do for different users and verify above privileges given)

```
SHOW GRANTS FOR 'jrc'@'localhost';
```

Revoke Privileges

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'jrc'@'localhost';
REVOKE ALL PRIVILEGES ON CollegeDB.* FROM 'gm'@'localhost';
REVOKE ALL PRIVILEGES ON CollegeDB.StuInfo FROM 'ram'@'localhost';
REVOKE INSERT ON CollegeDB.BookInfo FROM 'urmila'@'localhost'
REVOKE UPDATE,DELETE ON CollegeDB.BookBorrowInfo FROM 'urmila'@'localhost'
REVOKE CREATE ON *.* FROM 'laxman'@'localhost'
```

Again See list of privileges available for those users.

```
SHOW GRANTS FOR 'jrc'@'localhost'; // change username
```

Roles

A role in a database is a named group of privileges that can be assigned to one or more users. Instead of managing privileges individually for each user, roles simplify the process by grouping privileges under a single entity. Users can be granted or revoked roles, and the privileges associated with those roles will apply to the users.

Roles are essential for efficient database management, particularly in environments with multiple users and varying access requirements. Below are the reasons why roles are necessary:

1. Simplified Privilege Management

Without roles, privileges must be assigned individually to each user. Managing dozens or hundreds of users this way can become error-prone and inefficient.

With roles, privileges can be grouped and managed collectively, reducing administrative overhead.

Example: Instead of granting SELECT and INSERT privileges individually to 10 users, you can create a role data_entry with those privileges and assign the role to those users.

2. Consistency in Privilege Assignment

Roles ensure consistent access control by providing a standard set of privileges for users in similar roles or responsibilities.

This avoids discrepancies or oversights in granting permissions.

Example: All developers can be assigned a developer role that includes permissions to access development tables, reducing the risk of forgetting to grant necessary privileges.

3. Enhanced Security

Roles make it easier to enforce the principle of least privilege, ensuring users have only the permissions necessary for their work.

Sensitive operations can be restricted to specific roles, minimizing the risk of unauthorized access.

Example: A backup_operator role might only have access to database backup functions, ensuring regular users cannot perform backups.

4. Scalability

As organizations grow and the number of users increases, managing privileges becomes increasingly complex. Roles provide a scalable way to manage access.

Example: In a large organization, roles like HR_read_only or Finance_admin can be assigned to entire departments rather than individual users.

5. Easier Role Revocation

Revoking access becomes easier because you can revoke roles instead of managing individual privileges.

This is particularly useful when users leave the organization or change responsibilities.

Example: When a user moves from data entry to management, their data_entry role can be revoked, and a manager role can be assigned instead.

Real-World Scenarios

Multi-User Applications:

A web application might have roles like Admin, Editor, and Viewer, with varying privileges to manage content in the database.

Enterprise Systems:

ERP or CRM systems often use roles to control access for employees in different departments (e.g., Finance, Sales, HR).

Cloud Databases:

Cloud services like AWS RDS or Google Cloud SQL support roles to manage user access for database instances.

Examples :

Create a role developer :

CREATE ROLE developer;

Assign Privilege :

GRANT CREATE, ALTER, DROP, CREATE VIEW, CREATE ROUTINE,
CREATE TRIGGER, EXECUTE ON *.* TO 'developer';

Grant the role to users:

GRANT developer TO 'jrc'@'localhost';

GRANT developer TO 'drg'@'localhost';

Create a role DBA :

CREATE ROLE DBA;

Assign Privilege :

GRANT CREATE, ALTER, DROP, CREATE VIEW, CREATE ROUTINE,
CREATE TRIGGER, EXECUTE ON CollegeDB.* TO 'manager';

Grant the role to users:

GRANT DBA TO 'dba'@'localhost';

Create a role manager :

CREATE ROLE manager;

Assign Privilege :

GRANT SELECT, INSERT, UPDATE, DELETE ON CollegeDB.* TO 'manager';

Grant the role to users:

GRANT manager TO 'gm'@'localhost';

GRANT manager TO 'ram'@'localhost';

Create a role data_entry:

CREATE ROLE data_entry;

Assign Privilege :

GRANT SELECT, INSERT ON CollegeDB.* TO data_entry;

Grant the role to users :

GRANT data_entry TO 'urmila'@'localhost';

GRANT data_entry TO 'satrudhan'@'localhost';

Create a role receptionist :

CREATE ROLE receptionist;

Assign Privilege :

GRANT SELECT ON CollegeDB.StuInfo TO receptionist;

GRANT SELECT ON CollegeDB.EmplInfo TO receptionist;

GRANT SELECT ON CollegeDB.BookInfo TO receptionist;

Grant the role to users :

GRANT receptionist TO 'hanuman'@'localhost';

GRANT receptionist TO 'sugriv'@'localhost';

Create a role librarian :

CREATE ROLE librarian;

Assign Privilege :

GRANT SELECT ON CollegeDB.StuInfo TO librarian;

GRANT SELECT, INSERT ON CollegeDB.BookInfo TO librarian;

GRANT SELECT, INSERT, UPDATE, DELETE ON

CollegeDB.BookBorrowInfo TO librarian;

GRANT SELECT, INSERT, UPDATE, DELETE ON

CollegeDB.BookReturnInfo TO librarian;

Grant the role to users :

GRANT librarian TO 'libuser1'@'localhost';

REVOKE role From user and assign another role :

REVOKE receptionist FROM 'hanuman'@'localhost';

GRANT manager TO 'hanuman'@'localhost';

OR

SET DEFAULT ROLE manager FOR 'hanuman'@'localhost'; -- manager role automatically active when the user logs in.

.....

.....

Inserting Records in StuInfo Table :

INSERT INTO StuInfo (SName, Stream, SAddr, Semester, Gender, DOB, MobNo, Email)

VALUES

-- Example student records with diverse data

('Sita Rai', 'BCA', 'Biratnagar', 'IV', 'F', '2003-04-12', '9812345678', 'sita.ra@example.com'),
('Ram Bahadur', 'BBA', 'Kathmandu', 'III', 'M', '2002-09-20', '9801123456', 'ram.bahadur@example.com'),
('Rita Thapa', 'MCA', 'Pokhara', 'V', 'F', '2000-01-15', '9845671234', 'rita.thapa@example.com'),
('Kiran Sharma', 'MBA', 'Lalitpur', 'VII', 'M', '1999-11-08', '9810098765', 'kiran.sharma@example.com'),
('Puja Singh', 'BLLB', 'Dharan', 'VIII', 'F', '2001-03-25', '9802233445', 'puja.singh@example.com'),
('Nabin Gautam', 'BCA', 'Hetauda', 'II', 'M', '2003-06-15', '9843322110', 'nabin.gautam@example.com'),
('Anjali Tamang', 'BBA', 'Butwal', 'VI', 'F', '2001-08-20', '9804455667', 'anjali.tamang@example.com'),
('Bibek Lama', 'MCA', 'Chitwan', 'VIII', 'M', '1998-12-05', '9811223344', 'bibek.lama@example.com'),
('Rojina Karki', 'MBA', 'Janakpur', 'I', 'F', '2003-11-25', '9845566778', 'rojina.karki@example.com'),
('Suraj Khadka', 'BLLB', 'Nepalgunj', 'III', 'M', '2002-02-10', '9806677889', 'suraj.khadka@example.com');

Queries to Practice

- 1) List all the Students of College order by Stream, Semester and then alphabetically.
- 2) Display all the students of BCA IV semester
- 3) Retrieve all the female students with their Date of Birth
- 4) Display all the Students of MBA whose name starts with 'S'
- 5) Count all the students in the College
- 6) Count Total new students [ie. Students in I Semester]
- 7) Count all the Students in Each Stream and Semester
- 8) Count the total number male and female students in the College

.....

Inserting Records in FeelInfo Table

INSERT INTO FeelInfo (Sid, Semester, Amount, Paid)
VALUES

-- Student 1: Sita Rai (Semester IV)

(1, 'I', 35000.00, 'Y'),
(1, 'II', 36000.00, 'Y'),
(1, 'III', 37000.00, 'Y'),
(1, 'IV', 38000.00, 'Y'),

-- Student 2: Ram Bahadur (Semester III)

(2, 'I', 35000.00, 'Y'),
(2, 'II', 36000.00, 'Y'),
(2, 'III', 45000.00, 'N'),

-- Student 3: Rita Thapa (Semester V)

(3, 'I', 40000.00, 'Y'),
(3, 'II', 42000.00, 'Y'),
(3, 'III', 44000.00, 'Y'),
(3, 'IV', 46000.00, 'Y'),
(3, 'V', 55000.00, 'N'),

-- Student 4: Kiran Sharma (Semester VII)

(4, 'I', 40000.00, 'Y'),
(4, 'II', 42000.00, 'Y'),
(4, 'III', 44000.00, 'Y'),
(4, 'IV', 46000.00, 'Y'),
(4, 'V', 48000.00, 'Y'),
(4, 'VI', 50000.00, 'Y'),
(4, 'VII', 60000.00, 'N'),

-- Student 5: Puja Singh (Semester VIII)

(5, 'I', 35000.00, 'Y'),
(5, 'II', 36000.00, 'Y'),
(5, 'III', 37000.00, 'Y'),
(5, 'IV', 38000.00, 'Y'),
(5, 'V', 44000.00, 'Y'),
(5, 'VI', 46000.00, 'Y'),
(5, 'VII', 48000.00, 'Y'),
(5, 'VIII', 62000.00, 'N'),

-- Student 6: Nabin Gautam (Semester II)

(6, 'I', 35000.00, 'Y'),

(6, 'II', 40000.00, 'N'),

-- Student 7: Anjali Tamang (Semester VI)

(7, 'I', 38000.00, 'Y'),

(7, 'II', 40000.00, 'Y'),

(7, 'III', 42000.00, 'Y'),

(7, 'IV', 45000.00, 'Y'),

(7, 'V', 55000.00, 'Y'),

(7, 'VI', 58000.00, 'N'),

-- Student 8: Bibek Lama (Semester VIII)

(8, 'I', 37000.00, 'Y'),

(8, 'II', 38000.00, 'Y'),

(8, 'III', 40000.00, 'Y'),

(8, 'IV', 42000.00, 'Y'),

(8, 'V', 44000.00, 'Y'),

(8, 'VI', 46000.00, 'Y'),

(8, 'VII', 48000.00, 'Y'),

(8, 'VIII', 62000.00, 'N'),

-- Student 9: Rojina Karki (Semester I)

(9, 'I', 35000.00, 'Y'),

-- Student 10: Suraj Khadka (Semester III)

(10, 'I', 35000.00, 'Y'),

(10, 'II', 36000.00, 'Y'),

(10, 'III', 45000.00, 'N');

Queries Related to StuInfo and Fee Info Table

List of All Students with Unpaid Fees :

SELECT s.Sid, s.SName, f.Semester, f.Amount

FROM FeeInfo f

JOIN StuInfo s ON f.Sid = s.Sid

WHERE f.Paid = 'N';

Calculate Total Fees paid by a Student :

SELECT Sid, SUM(Amount) AS TotalPaid

FROM FeeInfo

WHERE Paid = 'Y'

GROUP BY Sid;

List of all students who have paid for all eligible semesters :

SELECT s.Sid, s.SName

FROM StuInfo s

LEFT JOIN FeeInfo f ON s.Sid = f.Sid

WHERE NOT EXISTS (

SELECT 1 FROM FeeInfo f1 WHERE f1.Sid = s.Sid AND f1.Paid = 'N'

);

3) BookInfo

CREATE TABLE BookInfo (

BookId INT AUTO_INCREMENT PRIMARY KEY,

Title VARCHAR(255) NOT NULL,

Genre VARCHAR(100),

Publisher VARCHAR(255),

Author VARCHAR(255) NOT NULL,

PublicationDate DATE,

ISBN VARCHAR(13) UNIQUE,

Pages INT CHECK (Pages > 0),

Language VARCHAR(25) DEFAULT 'English',

Price DECIMAL(8, 2) CHECK (Price >= 0)

);

INSERT INTO BookInfo (Title, Genre, Publisher, Author, PublicationDate,

ISBN, Pages, Language, Price)

VALUES

('Introduction to Programming', 'Technology', 'Oxford Nepal', 'John Doe', '2018-05-15', '9780198526636', 350, 'English', 1200.50),

('Basic Business Studies', 'Business', 'Ratna Pustak Bhandar', 'Hari Khadka', '2020-03-10', '9789937305172', 280, 'Nepali', 950.00),

('Advanced Mathematics', 'Education', 'Pilgrims Publishing', 'Bikash Karki', '2019-07-05', '9789937305202', 450, 'English', 1800.00),

('Legal Studies of Nepal', 'Law', 'Nepal Law Society', 'Sita Adhikari', '2021-02-20', '9789937305250', 320, 'Nepali', 1350.00),

('Computer Networks', 'Technology', 'Cambridge Press', 'James Smith', '2017-08-11', '9789937305289', 400, 'English', 1500.00),

('Python for Beginners', 'Programming', 'Packt', 'Daniel Liang', '2022-01-15', '9781788993373', 450, 'English', 2000.00),

('Nepali Governance', 'Law', 'Sajha Publications', 'Ram Bhandari', '2018-09-12', '9789937573184', 500, 'Nepali', 1100.00),

('Artificial Intelligence', 'Technology', 'O'Reilly', 'Stuart Russell', '2020-06-25',
'9781492032649', 600, 'English', 3000.00),
('Management Basics', 'Business', 'Penguin Nepal', 'Priya Sharma', '2019-03-05',
'9789937328257', 250, 'English', 800.00),
('Data Science Essentials', 'Education', 'Springer', 'Ravi Joshi', '2021-11-30',
'9783030326148', 350, 'English', 2500.00);

4) BookBorrowInfo

CREATE BookBorrowInfo(
BorrowId INT AUTO_INCREMENT PRIMARY KEY,
Sid INT,
BookId INT,
BorrowDate DATE DEFAULT CURRENT_DATE,
ToReturnDate DATE,
FOREIGN KEY (Sid) REFERENCES StuInfo(Sid),
FOREIGN KEY (BookId) REFERENCES BookInfo(BookId)
);

INSERT INTO BookBorrowInfo (Sid, BookId, BorrowDate, ToReturnDate)
VALUES

-- Student 1 (Sita Rai): Borrowed 2 books

(1, 1, '2024-01-05', '2024-01-20'),
(1, 2, '2024-01-07', '2024-01-22'),

-- Student 2 (Ram Bahadur): Borrowed 1 book

(2, 3, '2024-01-10', '2024-01-25'),

-- Student 3 (Rita Thapa): Borrowed 4 books

(3, 4, '2024-01-15', '2024-01-30'),
(3, 5, '2024-01-16', '2024-02-01'),
(3, 6, '2024-01-18', '2024-02-05'),
(3, 7, '2024-01-20', '2024-02-10'),

-- Student 4 (Kiran Sharma): Borrowed 5 books

(4, 8, '2024-01-21', '2024-02-15'),
(4, 9, '2024-01-23', '2024-02-17'),
(4, 10, '2024-01-25', '2024-02-20'),
(4, 1, '2024-01-27', '2024-02-22'),
(4, 2, '2024-01-29', '2024-02-25'),

-- Student 5 (Puja Singh): Did not borrow any books

-- Student 6 (Nabin Gautam): Borrowed 3 books

(6, 3, '2024-02-01', '2024-02-20'),
(6, 4, '2024-02-03', '2024-02-22'),
(6, 5, '2024-02-05', '2024-02-25'),

-- Student 7 (Anjali Tamang): Borrowed 5 books

(7, 6, '2024-02-06', '2024-02-28'),
(7, 7, '2024-02-08', '2024-03-02'),
(7, 8, '2024-02-10', '2024-03-05'),
(7, 9, '2024-02-12', '2024-03-08'),
(7, 10, '2024-02-14', '2024-03-10'),

-- Student 8 (Bibek Lama): Did not borrow any books

-- Student 9 (Rojina Karki): Borrowed 2 books

(9, 1, '2024-02-15', '2024-03-05'),
(9, 2, '2024-02-17', '2024-03-07'),

-- Student 10 (Suraj Khadka): Borrowed 1 book

(10, 3, '2024-02-20', '2024-03-10');